

1. 通信基本约定

本系统采用 Modbus-RTU 协议，物理层为 UART + RS485 半双工通信，波特率默认为2M。主机与从机采用严格一问一答方式通信，从机不主动发送数据。

通信链路为：



本系统使用的标准 Modbus 功能码如下：

```
#define MB_FC_READ_HOLDING_REGS      0x03  // 读保持寄存器
#define MB_FC_WRITE_SINGLE_REG      0x06  // 写单个寄存器
#define MB_FC_WRITE_MULTIPLE_REGS  0x10  // 写多个寄存器
```

1.1 功能码请求与回复约定

本系统使用标准 Modbus-RTU 一问一答格式。所有请求帧和回复帧末尾均带 CRC_L CRC_H，CRC 采用 Modbus-RTU CRC16，低字节在前。

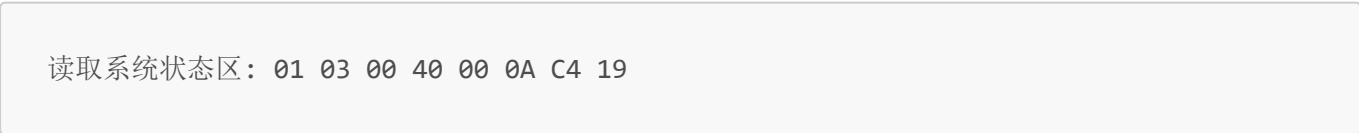
示例规则：除地址发现流程按协议使用 0x00 外，本文新增示例帧均固定从机地址为 0x01。变量数据区采用文中给出的业务示例值或全 0x00 示例值计算 CRC；[N bytes of 00] 表示压缩写法，实际发送或回复时应展开为 N 个 00 字节，数据变化后 CRC 必须重新计算。

1.1.1 0x03 读保持寄存器

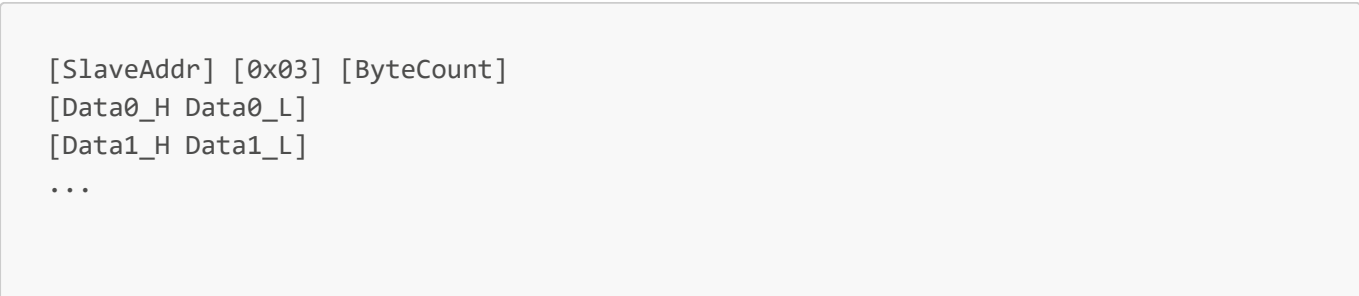
请求帧：



示例帧：



回复帧：



```
[DataN_H DataN_L]  
[CRC_L] [CRC_H]
```

示例帧：

1 个寄存器数据全 0: 01 03 02 00 00 B8 44

约定：

```
ByteCount = RegCount × 2  
N = RegCount - 1
```

1.1.2 0x06 写单个寄存器

请求帧：

```
[SlaveAddr] [0x06] [RegAddr_H RegAddr_L] [Value_H Value_L] [CRC_L] [CRC_H]
```

示例帧：

写 0x0001 = 0x0001: 01 06 00 01 00 01 19 CA

回复帧：

```
[SlaveAddr] [0x06] [RegAddr_H RegAddr_L] [Value_H Value_L] [CRC_L] [CRC_H]
```

示例帧：

写 0x0001 = 0x0001: 01 06 00 01 00 01 19 CA

约定：

0x06 的回复帧与请求帧中的寄存器地址和值保持一致。

1.1.3 0x10 写多个寄存器

请求帧：

```
[SlaveAddr] [0x10] [StartReg_H StartReg_L] [RegCount_H RegCount_L] [ByteCount]
[Data0_H Data0_L]
[Data1_H Data1_L]
...
[DataN_H DataN_L]
[CRC_L] [CRC_H]
```

示例帧：

写 REG_CMD/REG_CMD_PARAM/REG_CMD_SEQ: 01 10 00 20 00 03 06 00 94 00 00 00 01 17 F7

回复帧：

```
[SlaveAddr] [0x10] [StartReg_H StartReg_L] [RegCount_H RegCount_L] [CRC_L] [CRC_H]
```

示例帧：

写 0x0020 起始的 3 个寄存器回复: 01 10 00 20 00 03 81 C2

约定：

ByteCount = RegCount × 2
 N = RegCount - 1
 0x10 的回复帧只返回起始寄存器和写入寄存器数量，不回显数据区。

1.1.4 Modbus 异常回复

当请求帧格式正确、CRC 正确、从机地址匹配，但功能码、寄存器地址、寄存器数量或数据值不合法时，从机返回 Modbus 标准异常帧。

异常回复帧：

```
[SlaveAddr] [FuncCode | 0x80] [ExceptionCode] [CRC_L] [CRC_H]
```

示例帧：

0x10 异常码 0x03: 01 90 03 0C 01

常用异常码：

异常码	名称	使用场景
0x01	Illegal Function	不支持的功能码
0x02	Illegal Data Address	寄存器地址不存在，或访问范围越界
0x03	Illegal Data Value	寄存器数量、ByteCount、写入值不合法
0x04	Slave Device Failure	从机内部执行失败
0x06	Slave Device Busy	从机忙，暂时不能执行请求

示例：主机使用 0x10 写寄存器，但 ByteCount != RegCount × 2：

```
[SlaveAddr] [0x90] [0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 90 03 0C 01
```

不回复的情况：

- 1. CRC 校验失败；
- 2. 从机地址不匹配；
- 3. 帧长度小于 Modbus RTU 最小长度；
- 4. 总线冲突或接收溢出导致无法确认完整帧。

通信帧错误与命令执行错误分层处理：

Modbus 帧级错误：
返回异常帧，或 CRC/地址错误时不回复。

命令已成功写入 REG_CMD，但业务执行失败：
正常回复 0x10 写寄存器成功；
后续通过 REG_CMD_ACK / REG_CMD_ACK_SEQ / REG_CMD_ERROR 查询命令执行结果。

寄存器数据格式约定：

```
uint16:  
    占 1 个寄存器。  
  
float32:  
    占 2 个寄存器，采用小端寄存器序。
```

单个寄存器内部仍采用 Modbus 大端字节序。

uint64:

占 4 个寄存器，采用小端寄存器序。

单个寄存器内部仍采用 Modbus 大端字节序。

ros_timestamp:

占 4 个 Modbus 寄存器，共 8 byte。

seconds 字段为 32 bit，由 2 个连续的 16 bit Modbus 寄存器组成。

microseconds 字段为 32 bit，由 2 个连续的 16 bit Modbus 寄存器组成。

字段顺序固定为 seconds 在前，microseconds 在后。

每个 32 bit 字段内部采用小端寄存器序；单个寄存器内部仍采用 Modbus 大端字节序。

注意: ros_timestamp 不按 uint64 拼接，不表示“高 32 bit 为秒、低 32 bit 为微秒”的单个整数。

它是两个独立的 uint32 字段，解析后应得到 timestamp.sec 和 timestamp.usec。

如需计算绝对微秒数，应在上位机或固件中使用 uint64 计算：

```
total_us = (uint64)timestamp.sec * 1000000 + timestamp.usec
```

timestamp.usec 的合法范围为 0 ~ 999999。

例如：

```
raw64 = 0x1122334455667788
```

寄存器排列：

```
Reg + 0 = 0x7788
```

```
Reg + 1 = 0x5566
```

```
Reg + 2 = 0x3344
```

```
Reg + 3 = 0x1122
```

Modbus 数据区字节顺序：

```
77 88 55 66 33 44 11 22
```

ros_timestamp 示例：

```
timestamp.sec = 0x12345678
```

```
timestamp.usec = 0x000F423F // 999999 us
```

寄存器排列：

```
Reg + 0 = 0x5678 // seconds bit[15:0]
```

```
Reg + 1 = 0x1234 // seconds bit[31:16]
```

```
Reg + 2 = 0x423F // microseconds bit[15:0]
```

```
Reg + 3 = 0x000F // microseconds bit[31:16]
```

Modbus 数据区字节顺序：

```
56 78 12 34 42 3F 00 0F
```

解析方式：

```
timestamp.sec = ((uint32)Reg1 << 16) | Reg0
```

```
timestamp.usec = ((uint32)Reg3 << 16) | Reg2
```

不要按如下方式解释：

```
timestamp_u64 = ((uint64)seconds << 32) | microseconds
```

2. 地址发现任务

由于本系统为 1 对 1 通信，主机可使用地址 `0x00` 发送地址发现帧。该方式属于本系统在 1 对 1 场景下的约定用法。

2.1 主机发送地址发现帧

主机发送：

```
[0x00] [0x03] [0x00 0x00] [0x00 0x01] [CRC_L] [CRC_H]
```

示例帧：

```
00 03 00 00 00 01 85 DB
```

含义：

地址：0x00
功能码：0x03
起始寄存器：0x0000，即 REG_SLAVE_ADDR
寄存器数量：1

对应寄存器：

```
#define REG_SLAVE_ADDR 0x0000
```

2.2 从机使用 `0x00` 回复，并在数据区返回真实地址

假设从机真实地址为 `0x01`，从机回复帧的地址仍保持为主机请求中的 `0x00`，真实地址放在 REG_SLAVE_ADDR 数据区：

```
[0x00] [0x03] [0x02] [0x00 0x01] [CRC_L] [CRC_H]
```

示例帧：

```
00 03 02 00 01 44 44
```

主机解析寄存器值后记录：

```
slave_addr = 0x01;
```

后续所有通信均使用真实从机地址 `slave_addr`，不再使用 `0x00`。

3. 设备状态查询任务

地址发现完成后，主机使用真实从机地址读取各类状态寄存器。

3.1 读取基础通信与时间寄存器（初始化不管这里只是保留）

读取范围：

```
起始寄存器：0x0000
寄存器数量：14
```

请求帧：

```
[SlaveAddr] [0x03] [0x00 0x00] [0x00 0x0E] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 00 00 00 0E C4 0E
```

读取内容：

```
REG_SLAVE_ADDR      0x0000 // 从站地址
REG_BAUDRATE_CODE    0x0001 // 波特率编码
REG_UTC_TIMESTAMP    0x0002 // 当前系统 UTC 时间戳，ROS 格式，占 4 regs
REG_LOCAL_UPTIME_MS   0x0006 // 本地运行时间 us, uint64, 占 4 regs
REG_TIME_SYNC_UTC     0x000A // 主机写入同步 UTC，ROS 格式，占 4 regs
```

3.2 读取系统状态区

读取范围：

起始寄存器：0x0040
寄存器数量：10

请求帧：

[SlaveAddr] [0x03] [0x00 0x40] [0x00 0x0A] [CRC_L] [CRC_H]

示例帧：

01 03 00 40 00 0A C4 19

回复帧：

[SlaveAddr] [0x03] [0x14]
[Data0_H Data0_L]
[Data1_H Data1_L]
[Data2_H Data2_L]
[Data3_H Data3_L]
[Data4_H Data4_L]
[Data5_H Data5_L]
[Data6_H Data6_L]
[Data7_H Data7_L]
[Data8_H Data8_L]
[Data9_H Data9_L]
[CRC_L] [CRC_H]

示例帧：

01 03 14 [20 bytes of 00] A3 67

读取内容：

REG_SYSTEM_STATE	0x0040	// 系统状态
REG_WORK_MODE	0x0041	// 工作模式
REG_LOG_STATE	0x0042	// 日志状态
REG_SD_STATE	0x0043	// SD 卡状态
REG_SENSOR_STATE	0x0044	// 传感器状态位
REG_COMM_STATE	0x0045	// RS485 通信状态
REG_SYSTEM_RESERVED	0x0046	// 保留，0x0046 ~ 0x0047
REG_TEMPERATURE_BOARD	0x0048	// 板载温度，float32，占 2 regs

3.3 读取电源状态区（初始化不管这里只是保留）

读取范围：

起始寄存器：0x0060
寄存器数量：4

请求帧：

[SlaveAddr] [0x03] [0x00 0x60] [0x00 0x04] [CRC_L] [CRC_H]

示例帧：

01 03 00 60 00 04 44 17

回复帧：

[SlaveAddr] [0x03] [0x08]
[Data0_H Data0_L]
[Data1_H Data1_L]
[Data2_H Data2_L]
[Data3_H Data3_L]
[CRC_L] [CRC_H]

示例帧：

01 03 08 [8 bytes of 00] 95 D7

读取内容：

REG_BAT_VOLTAGE 0x0060 // 电池电压 V，float32，占 2 regs
REG_BAT_CURRENT 0x0062 // 电池电流 A，float32，占 2 regs

主机根据电池电压、电流判断是否存在异常：

电压过低：提示低电压或禁止开始采集；
电流过大：提示过流或禁止开启 SD 记录；
电流异常为 0：提示电流采样异常。

3.4 读取 SD 卡与日志状态区

读取范围：

起始寄存器：0x0081
寄存器数量：63

请求帧：

[SlaveAddr] [0x03] [0x00 0x81] [0x00 0x3F] [CRC_L] [CRC_H]

示例帧：

01 03 00 81 00 3F 55 F2

回复帧：

[SlaveAddr] [0x03] [0x7E]
[Data0_H Data0_L]
[Data1_H Data1_L]
...
[Data62_H Data62_L]
[CRC_L] [CRC_H]

示例帧：

01 03 7E [126 bytes of 00] 47 83

读取内容：

REG_SD_FS_STATUS	0x0081	// 文件系统状态
REG_SD_LOG_STATUS	0x0082	// 日志记录状态
REG_SD_ERROR_CODE	0x0083	// SD/FatFs 最近错误码

REG_SD_TOTAL_SIZE_MB	0x0084	// SD 卡总容量 MB, uint32
REG_SD_FREE_SIZE_MB	0x0086	// SD 卡剩余容量 MB, uint32
REG_SD_USED_SIZE_MB	0x0088	// SD 卡已用容量 MB, uint32
REG_SD_CURRENT_FILE_ID	0x008A	// 当前日志文件编号
REG_SD_CURRENT_FILE_SIZE	0x008C	// 当前日志文件大小 byte, uint64
REG_SD_CURRENT_WRITE_CNT	0x0090	// 当前文件已写入帧数, uint32
CMD_LOG_CREATE_FILE	0x0092	// 从机创建新的日志文件, 读取返回创建成功状态
REG_SD_RESERVED	0x0093	// 保留
REG_SD_DISK_LAST_RESULT	0x0094	// 最近一次 diskio DRESULT
REG_SD_DISK_HAL_STATUS	0x0095	// 最近一次 HAL_SD 操作返回状态
REG_SD_DISK_HAL_ERROR	0x0096	// 最近一次 HAL_SD 错误码, uint32, 占 2 regs: 0x0096 ~ 0x0097
REG_SD_RESERVED	0x0098 ~ 0x0099	
CMD_LOG_LENGTH	0x009A	// 查询指定文件长度, 读取返回文件长度, 占 4 regs
REG_SD_CURRENT_FILENAME	0x00A0	// 当前文件名
REG_SD_LAST_FILENAME	0x00B0	// 上一个文件名

注意：SD 状态区地址 0x0094 / 0x0096 是磁盘诊断寄存器；日志开始/停止不在 SD 状态区直接写寄存器触发，而是通过命令寄存器区 REG_CMD 0x0020 写入命令值 CMD_LOG_START 0x0094 / CMD_LOG_STOP 0x0096。

主机应检查：

- SD 文件系统是否已挂载；
- SD 是否存在错误；
- 剩余容量是否足够；
- 当前日志文件是否已打开；
- 当前写入帧数是否递增；
- 文件大小是否正常增长。

3.5 读取 IMU 状态

读取范围：

起始寄存器：0x1144
寄存器数量：1

请求帧：

[SlaveAddr] [0x03] [0x11 0x44] [0x00 0x01] [CRC_L] [CRC_H]

示例帧：

```
01 03 11 44 00 01 C1 23
```

回复帧：

```
[SlaveAddr] [0x03] [0x02]  
[Data0_H Data0_L]  
[CRC_L] [CRC_H]
```

示例帧：

```
01 03 02 [2 bytes of 00] B8 44
```

读取内容：

```
REG_IMU_STATUS_BITS      0x1144  // uint16, bit0~bit15 对应 IMU0~IMU15
```

状态位含义：

```
bit = 1: 对应 IMU 正常  
bit = 0: 对应 IMU 异常或离线
```

例如：

```
bit0  = IMU0 状态  
bit1  = IMU1 状态  
...  
bit15 = IMU15 状态
```

3.6 读取电阻点阵状态

读取范围：

```
起始寄存器：0x2088  
寄存器数量：9
```

请求帧：

[SlaveAddr] [0x03] [0x20 0x88] [0x00 0x09] [CRC_L] [CRC_H]

示例帧：

01 03 20 88 00 09 0E 26

回复帧：

[SlaveAddr] [0x03] [0x12]
[Data0_H Data0_L]
[Data1_H Data1_L]
[Data2_H Data2_L]
[Data3_H Data3_L]
[Data4_H Data4_L]
[Data5_H Data5_L]
[Data6_H Data6_L]
[Data7_H Data7_L]
[Data8_H Data8_L]
[CRC_L] [CRC_H]

示例帧：

01 03 12 [18 bytes of 00] F2 82

读取内容：

REG_R_STATUS_START 0x2088
REG_R_STATUS_END 0x2090

状态位分布：

0x2088: R0 ~ R15 状态位
0x2089: R16 ~ R31 状态位
0x208A: R32 ~ R47 状态位
0x208B: R48 ~ R63 状态位
0x208C: R64 ~ R79 状态位
0x208D: R80 ~ R95 状态位
0x208E: R96 ~ R111 状态位
0x208F: R112 ~ R127 状态位
0x2090: R128 ~ R131 状态位, bit4 ~ bit15 保留为 0

状态位含义：

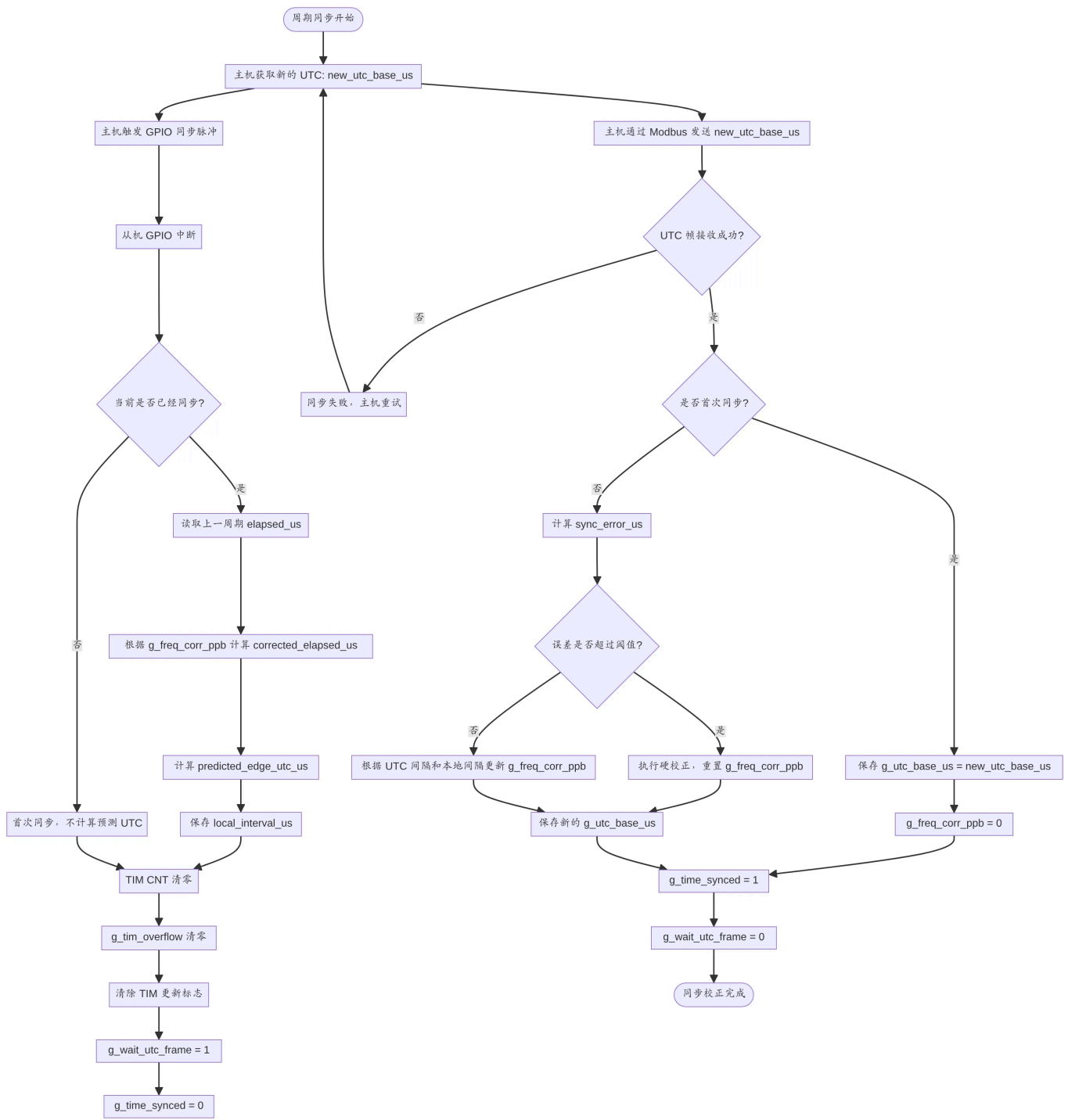
bit = 1: 对应电阻点正常

bit = 0: 对应电阻点异常、离线或数据无效

4. 时间同步任务

时间同步采用 GPIO 同步脉冲和 Modbus 写入 UTC 相结合的方式。

4.1 时间同步总体流程



1. 主机获取当前 UTC 时间 `new_utc_base_us`;
2. 主机触发 GPIO 同步脉冲;
3. 从机 GPIO 中断触发;
4. 从机在中断中执行:
 - 若当前已同步, 则读取上一周期 `elapsed_us`;
 - 根据 `g_freq_corr_ppb` 计算 `corrected_elapsed_us`;
 - 计算 `predicted_edge_utc_us`;
 - 保存 `local_interval_us`;
 - 清零 TIM CNT;
 - 清零 `g_tim_overflow`;
 - 清除 TIM 更新标志;
 - `g_wait_utc_frame = 1`;
 - `g_time_synced = 0`;
5. 主机通过 Modbus 写入 `new_utc_base_us`;
6. 从机收到 UTC 后判断是否首次同步;
7. 首次同步: 保存 `g_utc_base_us = new_utc_base_us`, `g_freq_corr_ppb = 0`;
8. 周期同步: 计算 `sync_error_us`, 并根据误差更新 `g_freq_corr_ppb`;
9. 设置 `g_time_synced = 1`;
10. 设置 `g_wait_utc_frame = 0`;
11. 同步完成。

4.2 主机写入同步 UTC

主机在 GPIO 同步脉冲发出后, 使用功能码 `0x10` 写入同步 UTC:

功能码: `0x10`
 起始寄存器: `0x000A`
 寄存器数量: 4

请求帧:

```
[SlaveAddr] [0x10] [0x00 0x0A] [0x00 0x04] [0x08]
[UTC_Reg0_H UTC_Reg0_L]
[UTC_Reg1_H UTC_Reg1_L]
[UTC_Reg2_H UTC_Reg2_L]
[UTC_Reg3_H UTC_Reg3_L]
[CRC_L] [CRC_H]
```

示例帧:

```
01 10 00 0A 00 04 08 60 00 0B B9 1A 83 00 06 42 18
```

回复帧:

```
[SlaveAddr] [0x10] [0x00 0x0A] [0x00 0x04] [CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 0A 00 04 E1 C8
```

其中：

```
UTC_Reg0 = bit[15:0]  
UTC_Reg1 = bit[31:16]  
UTC_Reg2 = bit[47:32]  
UTC_Reg3 = bit[63:48]
```

4.3 主机确认时间同步状态

主机随后读取当前系统 UTC 或系统状态：

```
读取当前 UTC：  
[SlaveAddr] [0x03] [0x00 0x02] [0x00 0x04] [CRC_L] [CRC_H]  
  
或者读取系统状态：  
[SlaveAddr] [0x03] [0x00 0x40] [0x00 0x0A] [CRC_L] [CRC_H]
```

示例帧：

```
读取当前 UTC：01 03 00 02 00 04 E5 C9  
或者读取系统状态：01 03 00 40 00 0A C4 19
```

读取当前 UTC 回复帧：

```
[SlaveAddr] [0x03] [0x08]  
[Data0_H Data0_L]  
[Data1_H Data1_L]  
[Data2_H Data2_L]  
[Data3_H Data3_L]  
[CRC_L] [CRC_H]
```

示例帧：


```
01 03 08 [8 bytes of 00] 95 D7
```

读取系统状态回复帧：

```
[SlaveAddr] [0x03] [0x14]  
[Data0_H Data0_L]  
[Data1_H Data1_L]  
[Data2_H Data2_L]  
[Data3_H Data3_L]  
[Data4_H Data4_L]  
[Data5_H Data5_L]  
[Data6_H Data6_L]  
[Data7_H Data7_L]  
[Data8_H Data8_L]  
[Data9_H Data9_L]  
[CRC_L] [CRC_H]
```

示例帧：

```
01 03 14 [20 bytes of 00] A3 67
```

主机判断同步是否成功：

若从机返回的 UTC 时间合理递增，则认为同步成功；
若超时、无响应或 UTC 不合理，则重新执行时间同步。

5. IMU 与关节角度校准任务

5.1 开始 IMU 与关节角度校准

IMU 校准不需要向从机发送校准命令。主机进入本地校准流程后，通过语音或界面提示用户动作，并持续读取 IMU 数据和关节角度数据用于计算 Offset 与动作状态。

5.2 主机语音提示用户执行动作

主机根据校准流程提示用户执行动作，例如：

动作 1：保持静止；
动作 2：手掌水平；
动作 3：向上弯曲；
动作 4：向下弯曲；

动作 5：握拳；
动作 6：张开。

在用户执行动作期间，主机持续读取 IMU 高频数据，用于判断动作是否稳定。

5.3 主机持续读取 IMU 高频数据

IMU 高频数据区：

0x1000 ~ 0x113F

拆成 3 帧读取：

```
Frame 0:
[SlaveAddr] [0x03] [0x10 0x00] [0x00 0x78] [CRC_L] [CRC_H]

Frame 1:
[SlaveAddr] [0x03] [0x10 0x78] [0x00 0x78] [CRC_L] [CRC_H]

Frame 2:
[SlaveAddr] [0x03] [0x10 0xF0] [0x00 0x50] [CRC_L] [CRC_H]
```

示例帧：

```
Frame 0: 01 03 10 00 00 78 41 28
Frame 1: 01 03 10 78 00 78 C1 31
Frame 2: 01 03 10 F0 00 50 41 05
```

对应：

```
Frame 0: IMU float[0] ~ float[59]
Frame 1: IMU float[60] ~ float[119]
Frame 2: IMU float[120] ~ float[159]
```

5.4 关节角度校准说明

关节角度由 IMU 姿态解算得到，不直接来自独立角度传感器。因此关节角度校准的对象不是“关节角度数据本身”，而是 IMU 到关节角度解算链路中的零点、安装偏差和映射参数。

误差来源主要包括：

1. IMU 自身零偏，例如陀螺仪零偏、加速度计零偏；
2. IMU 安装方向偏差，即传感器坐标系与手指/手掌坐标系不完全一致；
3. 初始姿态零点偏差，例如五指自然伸直时各关节应定义为 0 度或标准初始角度；
4. 关节角度映射模型误差，即多个 IMU 姿态转换为 21 个关节角度时的模型偏差；
5. 个体差异和佩戴差异，例如手型、绑带松紧和传感器位置变化。

建议校准流程：

1. 先完成 IMU 姿态/零偏校准；
2. 用户保持标准初始手势，例如手掌平放、五指自然伸直；
3. 主机读取当前解算出的 21 个关节角度 `joint\_angle\_raw[21]`；
4. 根据标准手势期望角度 `expected\_angle[21]` 计算零点偏移；
5. 运行时输出校准后的关节角度。

基础零点校准公式：

```
joint_angle_offset[i] = joint_angle_raw[i] - expected_angle[i]
joint_angle[i] = joint_angle_raw[i] - joint_angle_offset[i]
```

若后续需要更高精度，可增加比例系数或非线性映射：

```
joint_angle[i] = scale[i] * joint_angle_raw[i] + offset[i]
```

第一版建议先采用 offset 零点校准，简单可靠，便于调试和现场复现。

5.5 关节角度校准值寄存器区

关节角度相关寄存器统一从 0x1E00 开始排列，顺序为：关节角度、ROS 时间戳、关节角度状态、校准后的关节角度校准值。

```
0x1E00 ~ 0x1E29: joint_angle[0] ~ joint_angle[20], float32, 共 42 regs
0x1E2A ~ 0x1E2D: joint_timestamp, ROS 格式时间戳, 共 4 regs
    0x1E2A: seconds bit[15:0]
    0x1E2B: seconds bit[31:16]
    0x1E2C: microseconds bit[15:0]
    0x1E2D: microseconds bit[31:16]
0x1E2E ~ 0x1E2F: joint_angle_status, 当前先用 2 个寄存器表示 21 个关节角度状态
0x1E30 ~ 0x1E59: joint_angle_calib[0] ~ joint_angle_calib[20], float32, 共 42 regs
```

其中 joint_timestamp 是两个独立的 uint32 字段，不按单个 uint64 拼接；详见 1.1.4 中的 ros_timestamp 约定。

读取范围：

起始寄存器：0x1E30
寄存器数量：42

请求帧：

[SlaveAddr] [0x03] [0x1E 0x30] [0x00 0x2A] [CRC_L] [CRC_H]

示例帧：

01 03 1E 30 00 2A C2 32

对应：

0x1E30 ~ 0x1E59: joint_angle_calib[0] ~ joint_angle_calib[20], float32, 共 42 regs

保存关节角度校准值时，主机可使用 0x10 写多个寄存器：

[SlaveAddr] [0x10] [0x1E 0x30] [0x00 0x2A] [0x54]
[CalibReg0_H CalibReg0_L]
[CalibReg1_H CalibReg1_L]
...
[CalibReg41_H CalibReg41_L]
[CRC_L] [CRC_H]

示例帧：

数据区全 0: 01 10 1E 30 00 2A 54 [84 bytes of 00] 04 13

从机运行时根据该校准值输出校准后的关节角度：

$\text{joint_angle}[i] = \text{joint_angle_raw}[i] - \text{joint_angle_calib}[i]$

5.6 关节角度数据寄存器区

关节角度数据区起始寄存器为 0x1E00，用于返回从机运行时输出的 21 个关节角度。

读取范围：

起始寄存器：0x1E00
寄存器数量：42

请求帧：

[SlaveAddr] [0x03] [0x1E 0x00] [0x00 0x2A] [CRC_L] [CRC_H]

示例帧：

01 03 1E 00 00 2A C2 3D

对应：

0x1E00 ~ 0x1E29: joint_angle[0] ~ joint_angle[20], float32, 共 42 regs

关节角度数据是运行时输出值，主机不通过该区域写入校准参数；校准参数应写入 0x1E30 ~ 0x1E59 的关节角度校准值区。

5.7 记录 IMU 校准步骤数据

每个动作完成后，主机在本地记录当前动作对应的 IMU 数据快照或统计结果，例如静止均值、姿态初始值、加速度零偏和角速度零偏。

从机只负责返回实时 IMU 数据，不需要接收校准步骤命令，也不需要返回校准 ACK。

5.8 保存 IMU 校准参数

所有动作完成后，主机根据采样结果计算每个 IMU 的 Offset，并通过功能码 0x10 写入 IMU Offset 寄存器区：

起始寄存器：0x1154
寄存器数量：320

由于单帧写多个寄存器不宜过长，建议按单个 IMU 分 16 帧写入。单个 IMU Offset 写入范围：

起始寄存器：IMU_i_OFFSET_BASE = 0x1154 + i × 20
寄存器数量：20

数据长度: 40 byte
i = 0 ~ 15

单个 IMU Offset 写入请求帧:

```
[SlaveAddr] [0x10] [IMU_i_OFFSET_BASE_H IMU_i_OFFSET_BASE_L] [0x00 0x14] [0x28]  
[OffsetReg0_H OffsetReg0_L]  
[OffsetReg1_H OffsetReg1_L]  
...  
[OffsetReg19_H OffsetReg19_L]  
[CRC_L] [CRC_H]
```

示例帧:

i = 0, Offset 数据全 0: 01 10 11 54 00 14 28 [40 bytes of 00] 13 6F

写入成功后, 从机按标准 0x10 回复:

```
[SlaveAddr] [0x10] [IMU_i_OFFSET_BASE_H IMU_i_OFFSET_BASE_L] [0x00 0x14] [CRC_L]  
[CRC_H]
```

示例帧:

i = 0: 01 10 11 54 00 14 84 EA

6. 电阻点阵校准任务

6.1 开始电阻点阵校准

电阻点阵校准不需要向从机发送校准命令。主机进入本地校准流程后, 提示用户保持无压力状态, 并通过读取电阻点阵 ADC 原始值计算零点基准。

6.2 零点校准

用户保持无压力状态, 主机连续读取电阻点阵 ADC 原始值并在本地计算 132 个电阻点的零点基准。

6.3 主机读取电阻点阵 ADC 原始值

电阻点阵 ADC 原始值区:

0x2000 ~ 0x2083

单独读取电阻点阵校准数据时可拆成 3 帧读取：

```
Frame 0:
[SlaveAddr] [0x03] [0x20 0x00] [0x00 0x3C] [CRC_L] [CRC_H]

Frame 1:
[SlaveAddr] [0x03] [0x20 0x3C] [0x00 0x3C] [CRC_L] [CRC_H]

Frame 2:
[SlaveAddr] [0x03] [0x20 0x78] [0x00 0x0C] [CRC_L] [CRC_H]
```

示例帧：

```
Frame 0: 01 03 20 00 00 3C 4E 1B
Frame 1: 01 03 20 3C 00 3C 8E 17
Frame 2: 01 03 20 78 00 0C CE 16
```

对应：

```
Frame 0: R_ADC[0] ~ R_ADC[59]
Frame 1: R_ADC[60] ~ R_ADC[119]
Frame 2: R_ADC[120] ~ R_ADC[131]
```

6.4 保存电阻点阵校准参数

主机在本地保存 132 个电阻点的 ADC 零点基准。当前文档未定义电阻点阵 Offset 寄存器区，因此本流程不发送保存命令。

7. 开启 SD 记录任务

7.1 创建日志文件

主机通过功能码 0x10 写入 CMD_LOG_CREATE_FILE，触发从机创建新的日志文件。

```
功能码：0x10
起始寄存器：0x0092
寄存器数量：1
写入值：1
```

请求帧：

```
[SlaveAddr] [0x10] [0x00 0x92] [0x00 0x01] [0x02]  
[0x00 0x01]  
[CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 92 00 01 02 00 01 7B 22
```

回复帧：

```
[SlaveAddr] [0x10] [0x00 0x92] [0x00 0x01] [CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 92 00 01 A0 24
```

从机创建新的日志文件，并更新：

```
REG_SD_CURRENT_FILE_ID  
REG_SD_CURRENT_FILENAME  
REG_SD_CURRENT_FILE_SIZE  
REG_SD_LOG_STATUS
```

主机读取 SD 状态区确认文件是否创建成功：

```
[SlaveAddr] [0x03] [0x00 0x81] [0x00 0x3F] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 00 81 00 3F 55 F2
```

7.2 开始日志记录

主机通过命令寄存器写入 `CMD_LOG_START`，触发从机开始日志记录。


```
REG_CMD      0x0020 = 0x0094  // CMD_LOG_START
REG_CMD_PARAM 0x0021 = 0
REG_CMD_SEQ   0x0022 = seq
```

请求帧：

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [0x06]
[0x00 0x94]
[0x00 0x00]
[SEQ_H SEQ_L]
[CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 20 00 03 06 00 94 00 00 00 01 17 F7
```

回复帧：

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 20 00 03 81 C2
```

主机读取 ACK：

```
[SlaveAddr] [0x03] [0x00 0x23] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 00 23 00 03 F4 01
```

从机开始将 IMU、电阻点阵、时间戳、电池状态等数据写入 SD 卡。

主机读取：

```
[SlaveAddr] [0x03] [0x00 0x82] [0x00 0x01] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 00 82 00 01 24 22
```

确认：

```
REG_SD_LOG_STATUS = 正在记录
```

7.3 停止日志记录

主机通过命令寄存器写入 **CMD_LOG_STOP**，触发从机停止日志记录。

```
REG_CMD      0x0020 = 0x0096  // CMD_LOG_STOP  
REG_CMD_PARAM 0x0021 = 0  
REG_CMD_SEQ   0x0022 = seq
```

请求帧：

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [0x06]  
[0x00 0x96]  
[0x00 0x00]  
[SEQ_H SEQ_L]  
[CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 20 00 03 06 00 96 00 00 00 02 2E 36
```

回复帧：

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 20 00 03 81 C2
```

主机读取 ACK:

```
[SlaveAddr] [0x03] [0x00 0x23] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧:

```
01 03 00 23 00 03 F4 01
```

主机读取日志状态确认:

```
[SlaveAddr] [0x03] [0x00 0x82] [0x00 0x01] [CRC_L] [CRC_H]
```

示例帧:

```
01 03 00 82 00 01 24 22
```

确认:

```
REG_SD_LOG_STATUS = 停止记录
```

7.4 读取 SD 文件列表 (暂未实现)

主机通过命令寄存器请求从机准备 SD 文件列表。文件列表按页读取，`REG_CMD_PARAM` 表示 `page_id`，从 0 开始。

```
REG_CMD      0x0020 = TBD      // CMD_SD_LIST_FILES, 命令字待重新分配
REG_CMD_PARAM 0x0021 = page_id
REG_CMD_SEQ   0x0022 = seq
```

请求帧:

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [0x06]
[CMD_SD_LIST_FILES_H CMD_SD_LIST_FILES_L]
[PageId_H PageId_L]
```

```
[SEQ_H SEQ_L]
[CRC_L] [CRC_H]
```

示例说明：**CMD_SD_LIST_FILES** 命令字尚未分配，请求帧 CRC 需在命令字确定后重算。

回复帧：

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 20 00 03 81 C2
```

主机读取 ACK：

```
[SlaveAddr] [0x03] [0x00 0x23] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 00 23 00 03 F4 01
```

若 ACK 成功，则从机已准备好该页文件列表。文件列表数据区需在后续协议中补充定义；当前命令仅定义触发方式和参数。

7.5 ~~下载指定日志文件（暂未实现）~~

主机通过命令寄存器请求从机准备指定日志文件下载。**REG_CMD_PARAM** 表示 **file_id**。

```
REG_CMD      0x0020 = TBD      // CMD_SD_DOWNLOAD_FILE，命令字待重新分配
REG_CMD_PARAM 0x0021 = file_id
REG_CMD_SEQ   0x0022 = seq
```

请求帧：

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [0x06]  
[CMD_SD_DOWNLOAD_FILE_H CMD_SD_DOWNLOAD_FILE_L]  
[FileId_H FileId_L]  
[SEQ_H SEQ_L]  
[CRC_L] [CRC_H]
```

示例说明：**CMD_SD_DOWNLOAD_FILE** 命令字尚未分配，请求帧 CRC 需在命令字确定后重算。

回复帧：

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 20 00 03 81 C2
```

主机读取 ACK：

```
[SlaveAddr] [0x03] [0x00 0x23] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 00 23 00 03 F4 01
```

若 ACK 成功，则从机已准备好指定文件下载。文件下载数据区、分片大小和分片读取寄存器需在后续协议中补充定义。

7.6 SD 卡数据块存储规则

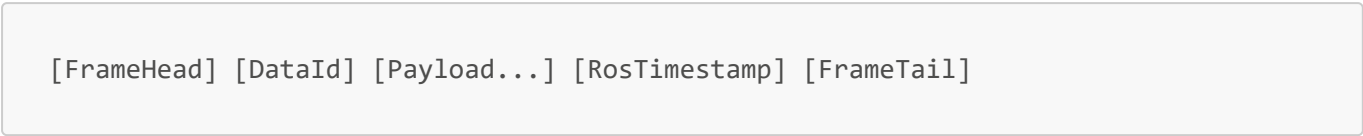
SD 卡记录采用固定 **1024 byte** 数据块。每个数据块由 3 个业务子帧、**CRC16** 和末尾分隔符组成。

采用 **1024 byte** 固定块的原因：

1. SD 卡常见物理扇区大小为 512 byte；
2. 1 帧完整采集数据固定为 1024 byte，刚好等于 2 个扇区；
3. 写入时天然按扇区对齐，便于 FatFs/底层块设备连续写入；
4. 固定长度记录便于掉电后按块扫描、CRC 校验和快速定位损坏帧；
5. 每帧长度固定后，文件偏移可直接通过 $\text{frame_index} \times 1024$ 计算。

7.6.1 子帧格式

每个业务子帧格式如下：



字段说明：

字段	长度	说明
FrameHead	1 byte	固定为 0xA5
DataId	1 byte	数据标识符，区分左右手和数据类型
Payload	N byte	数据载荷
RosTimestamp	8 byte	ROS 时间戳：seconds uint32 + microseconds uint32
FrameTail	1 byte	固定为 0x5A

7.6.2 数据标识符

右手标识符：

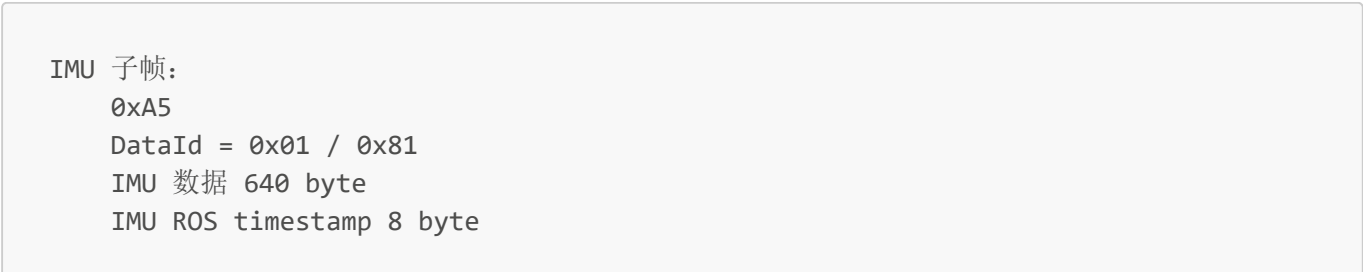
数据类型	标识符
IMU 数据	0x01
关节角度	0x02
电阻点阵 / 触觉 ADC 原始值	0x03

左手标识符为右手标识符加 0x80：

数据类型	标识符
IMU 数据	0x81
关节角度	0x82
电阻点阵 / 触觉 ADC 原始值	0x83

7.6.3 固定 1024 byte 数据块布局

单个 SD 数据块布局如下：



0x5A

关节角度子帧:

0xA5

DataId = 0x02 / 0x82

关节角度数据 84 byte

joint ROS timestamp 8 byte

0x5A

触觉数据子帧:

0xA5

DataId = 0x03 / 0x83

触觉 / 电阻点阵 ADC 原始值 264 byte

tactile ROS timestamp 8 byte

0x5A

块尾:

CRC16_L

CRC16_H

Separator = 0x00

长度计算:

IMU 子帧: $1 + 1 + 640 + 8 + 1 = 651$ byte

关节角度子帧: $1 + 1 + 84 + 8 + 1 = 95$ byte

触觉数据子帧: $1 + 1 + 264 + 8 + 1 = 275$ byte

业务数据合计: $651 + 95 + 275 = 1021$ byte

CRC16: 2 byte

分隔符: 1 byte

SD 数据块总长度: $1021 + 2 + 1 = 1024$ byte

7.6.4 数据拷贝方式与字节序

SD 卡数据块中的所有业务数据均通过 `memcpy` 从内存直接拷贝到写入缓冲区, 不做逐字段字节翻转。

IMU float32[160]:

memcpy 640 byte

关节角度 float32[21]:

memcpy 84 byte

触觉 / 电阻点阵 ADC uint16[132]:

memcpy 264 byte

ROS timestamp:

```
memcpy 8 byte
```

其中前 4 byte 为 seconds uint32, 后 4 byte 为 microseconds uint32

SD 卡记录保存的是运行时关节角度数据; 0x1E30 ~ 0x1E59 的关节角度校准值仅作为校准参数区, 不直接作为 SD 关节角度子帧写入。

本系统 MCU 为小端序, 因此 SD 卡文件中的 float32、uint16、uint32 均按小端内存布局保存。

解析 SD 卡文件时, 上位机应按小端序还原数据:

float32: 低地址字节为最低有效字节

uint16: 低地址字节为最低有效字节

uint32: 低地址字节为最低有效字节

示例代码:

```
#include <stdint.h>
#include <string.h>

#define SD_LOG_BLOCK_SIZE      1024U
#define SD_LOG_CONTENT_SIZE    1021U
#define SD_LOG_FRAME_HEAD     0xA5U
#define SD_LOG_FRAME_TAIL     0x5AU
#define SD_LOG_SEPARATOR      0x00U

#define SD_LOG_ID_RIGHT_IMU    0x01U
#define SD_LOG_ID_RIGHT_JOINT   0x02U
#define SD_LOG_ID_RIGHT_TACTILE 0x03U
#define SD_LOG_ID_LEFT_IMU     0x81U
#define SD_LOG_ID_LEFT_JOINT    0x82U
#define SD_LOG_ID_LEFT_TACTILE 0x83U

typedef struct
{
    uint32_t sec;
    uint32_t usec;
} SdLog_RosTimestamp_t;

static uint16_t SdLog_Crc16(const uint8_t *data, uint32_t len)
{
    uint16_t crc = 0xFFFFU;

    for (uint32_t i = 0U; i < len; i++)
    {
        crc ^= data[i];
        for (uint8_t bit = 0U; bit < 8U; bit++)
        {
            if ((crc & 0x0001U) != 0U)
            {

```



```

        crc = (uint16_t)((crc >> 1) ^ 0xA001U);
    }
    else
    {
        crc >>= 1;
    }
}

return crc;
}

static uint32_t SdLog_AppendSubFrame(uint8_t *block,
                                     uint32_t offset,
                                     uint8_t data_id,
                                     const void *payload,
                                     uint32_t payload_len,
                                     const SdLog_RosTimestamp_t *timestamp)
{
    block[offset++] = SD_LOG_FRAME_HEAD;
    block[offset++] = data_id;

    memcpy(&block[offset], payload, payload_len);
    offset += payload_len;

    memcpy(&block[offset], timestamp, sizeof(*timestamp));
    offset += (uint32_t)sizeof(*timestamp);

    block[offset++] = SD_LOG_FRAME_TAIL;

    return offset;
}

void SdLog_BuildBlock(uint8_t block[SD_LOG_BLOCK_SIZE],
                     uint8_t is_left_hand,
                     const float imu_data[160],
                     SdLog_RosTimestamp_t imu_timestamp,
                     const float joint_angle[21],
                     SdLog_RosTimestamp_t joint_timestamp,
                     const uint16_t tactile_adc[132],
                     SdLog_RosTimestamp_t tactile_timestamp)
{
    uint32_t offset = 0U;
    uint16_t crc;
    uint8_t imu_id = is_left_hand ? SD_LOG_ID_LEFT_IMU : SD_LOG_ID_RIGHT_IMU;
    uint8_t joint_id = is_left_hand ? SD_LOG_ID_LEFT_JOINT : SD_LOG_ID_RIGHT_JOINT;
    uint8_t tactile_id = is_left_hand ? SD_LOG_ID_LEFT_TACTILE :
SD_LOG_ID_RIGHT_TACTILE;

    offset = SdLog_AppendSubFrame(block,
                                  offset,
                                  imu_id,
                                  imu_data,
                                  160U * sizeof(float),

```

```
        &imu_timestamp);

    offset = SdLog_AppendSubFrame(block,
                                   offset,
                                   joint_id,
                                   joint_angle,
                                   21U * sizeof(float),
                                   &joint_timestamp);

    offset = SdLog_AppendSubFrame(block,
                                   offset,
                                   tactile_id,
                                   tactile_adc,
                                   132U * sizeof(uint16_t),
                                   &tactile_timestamp);

    /* offset should be 1021 here. */
    crc = SdLog_Crc16(block, SD_LOG_CONTENT_SIZE);
    block[offset++] = (uint8_t)(crc & 0xFFU);
    block[offset++] = (uint8_t)(crc >> 8);
    block[offset++] = SD_LOG_SEPARATOR;
}
```

CRC 采用 Modbus RTU CRC16 算法，低字节在前。CRC 覆盖范围为前 1021 byte，即 3 个完整业务子帧；不包含最后的 CRC16_L CRC16_H 和 Separator。

末尾分隔符固定为：

```
Separator = 0x00
```

7.6.5 SD 卡容量估算

按高频采集 100 Hz 计算：

```
单帧大小：1024 byte
采样频率：100 frame/s
每秒数据量：1024 × 100 = 102400 byte/s
每小时数据量：102400 × 3600 = 368640000 byte ≈ 351.56 MiB
每天 8 小时数据量：368640000 × 8 = 2949120000 byte ≈ 2.95 GB ≈ 2.75 GiB
```

按 SD 卡厂家常用十进制容量估算：

SD 卡容量	每天 8 小时采集	理论可保存时长
16 GB	约 2.95 GB/day	约 5.4 天
32 GB	约 2.95 GB/day	约 10.8 天

考虑文件系统元数据、坏块预留、实际可用容量和日志索引开销，工程上建议按以下保守值规划：

SD 卡容量	建议按可用时长
16 GB	约 5 天
32 GB	约 10 天

8. 采集控制任务

采集控制统一使用命令寄存器区下发。主机通过 `REG_CMD` 写入采集命令字，从机执行后更新 `REG_WORK_STATE`，主机通过读取 `REG_WORK_STATE` 确认采集状态。

采集控制相关定义：

```
#define REG_WORK_STATE 0x0500 // 工作状态
#define CMD_ACQ_START 0x0501 // 开始采集任务
#define CMD_ACQ_STOP 0x0502 // 结束采集任务
```

工作状态建议值：

```
#define WORK_STATE_IDLE 0x0000 // 空闲
#define WORK_STATE_ACQUIRING 0x0001 // 正在采集
#define WORK_STATE_STOPPING 0x0002 // 正在停止采集
#define WORK_STATE_ERROR 0x8000 // 采集异常
```

8.1 开始采集任务

主机发送开始采集命令：

```
REG_CMD      0x0020 = 0x0501 // CMD_ACQ_START
REG_CMD_PARAM 0x0021 = 0
REG_CMD_SEQ   0x0022 = seq
```

请求帧：

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [0x06]
[0x05 0x01]
[0x00 0x00]
[SEQ_H SEQ_L]
[CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 20 00 03 06 05 01 00 00 00 03 9A 7E
```

回复帧：

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 20 00 03 81 C2
```

主机读取 ACK：

```
[SlaveAddr] [0x03] [0x00 0x23] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 00 23 00 03 F4 01
```

从机收到后：

1. 启动 IMU 采集任务；
2. 启动电阻点阵采集任务；
3. 更新 IMU 时间戳；
4. 更新电阻点阵时间戳；
5. 更新 `REG\_WORK\_STATE = WORK\_STATE\_ACQUIRING`；
6. 准备响应主机 100 Hz 高频读取。

主机读取工作状态确认：

```
[SlaveAddr] [0x03] [0x05 0x00] [0x00 0x01] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 05 00 00 01 84 C6
```

若返回 `REG_WORK_STATE = WORK_STATE_ACQUIRING`，则进入高频数据交互阶段。

8.2 结束采集任务

主机发送结束采集命令：

```
REG_CMD      0x0020 = 0x0502  // CMD_ACQ_STOP  
REG_CMD_PARAM 0x0021 = 0  
REG_CMD_SEQ   0x0022 = seq
```

请求帧：

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [0x06]  
[0x05 0x02]  
[0x00 0x00]  
[SEQ_H SEQ_L]  
[CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 20 00 03 06 05 02 00 00 00 04 9F BC
```

回复帧：

```
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 10 00 20 00 03 81 C2
```

主机读取 ACK：

```
[SlaveAddr] [0x03] [0x00 0x23] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 00 23 00 03 F4 01
```

从机收到后：

1. 停止 IMU 高频采集任务或切换为空闲采样状态；
 2. 停止电阻点阵高频采集任务或切换为空闲采样状态；
 3. 停止响应主机 100 Hz 高频读取流程；
 4. 更新 `REG\_WORK\_STATE = WORK\_STATE\_IDLE`；
 5. 保留最后一次 IMU、电阻点阵和时间戳数据，供主机低频读取确认；
 6. 如 SD 日志正在记录，则根据系统策略继续记录或等待日志停止命令，不由 `CMD\_ACQ\_STOP` 隐式关闭日志。

主机读取工作状态确认：

[SlaveAddr] [0x03] [0x05 0x00] [0x00 0x01] [CRC_L] [CRC_H]

示例帧：

01 03 05 00 00 01 84 C6

若返回 `REG_WORK_STATE = WORK_STATE_IDLE`，则退出高频数据交互阶段。

9. 高频数据交互任务

采集任务开启后，主机以 100 Hz 周期读取 7 帧数据。

9.1 高频数据内容

IMU 数据：

- 160 个 float
- 320 个寄存器
- 640 byte

IMU 时间戳：

- ROS 格式：seconds uint32 + microseconds uint32
- 4 个寄存器
- 8 byte

关节角度数据：

- 21 个 float
- 42 个寄存器
- 84 byte

关节角度时间戳：

- ROS 格式：seconds uint32 + microseconds uint32
- seconds 字段由 2 个 16 bit 寄存器组成
- microseconds 字段由 2 个 16 bit 寄存器组成
- 4 个寄存器
- 8 byte

电阻点阵 ADC 原始值:

132 个 uint16
132 个寄存器
264 byte

电阻点阵时间戳:

ROS 格式: seconds uint32 + microseconds uint32
4 个寄存器
8 byte

总计:

506 个寄存器
1012 byte

9.2 高频数据 7 帧读取表

```
typedef struct
{
    uint16_t start_reg;
    uint16_t reg_count;
} ModbusReadSegment_t;

static const ModbusReadSegment_t g_high_rate_segments[7] =
{
    {0x1000, 120}, // IMU float[0] ~ float[59]
    {0x1078, 120}, // IMU float[60] ~ float[119]
    {0x10F0, 84},  // IMU float[120] ~ float[159] + ROS timestamp
    {0x1E00, 46},  // Joint angle[0] ~ angle[20] + ROS timestamp
    {0x2000, 60},  // R_ADC[0] ~ R_ADC[59]
    {0x203C, 60},  // R_ADC[60] ~ R_ADC[119]
    {0x2078, 16},  // R_ADC[120] ~ R_ADC[131] + ROS timestamp
};
```

9.3 高频读取请求帧

采集任务开始后, 主机以 100 Hz 周期读取 7 帧高频数据。读取功能码统一为 0x03, 即读取保持寄存器。

标准 Modbus 0x03 响应帧格式如下:

```
[SlaveAddr] [0x03] [ByteCount] [Data...] [CRC_L] [CRC_H]
```

示例帧:

1 个寄存器数据全 0: 01 03 02 00 00 B8 44

其中：

$$\text{ByteCount} = \text{RegisterCount} \times 2$$

Frame 0: 读取 IMU float[0] ~ float[59]

请求帧：

```
[SlaveAddr] [0x03] [0x10 0x00] [0x00 0x78] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 10 00 00 78 41 28
```

含义：

起始寄存器：0x1000
寄存器数量：0x0078 = 120
数据长度：120 × 2 = 240 byte = 0xF0
对应数据：60 个 float32

回复帧：

```
[SlaveAddr] [0x03] [0xF0]  
[Reg1000_H Reg1000_L]  
[Reg1001_H Reg1001_L]  
...  
[Reg1077_H Reg1077_L]  
[CRC_L] [CRC_H]
```

示例帧：

```
01 03 F0 [240 bytes of 00] 8C DB
```

对应 IMU 数据范围：

```
IMU float[0] ~ IMU float[59]
```


其中每个 float32 占 2 个寄存器，采用小端寄存器序。例如某个 float 的原始数据为：

```
raw32 = 0xAABBCCDD
```

则响应数据区对应：

```
Reg + 0 = 0xCCDD
```

```
Reg + 1 = 0xAABB
```

字节顺序：

```
CC DD AA BB
```

Frame 1：读取 IMU float[60] ~ float[119]

请求帧：

```
[SlaveAddr] [0x03] [0x10 0x78] [0x00 0x78] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 10 78 00 78 C1 31
```

含义：

起始寄存器：0x1078

寄存器数量：0x0078 = 120

数据长度：120 × 2 = 240 byte = 0xF0

对应数据：60 个 float32

回复帧：

```
[SlaveAddr] [0x03] [0xF0]
```

```
[Reg1078_H Reg1078_L]
```

```
[Reg1079_H Reg1079_L]
```

```
...
```

```
[Reg10EF_H Reg10EF_L]
```

```
[CRC_L] [CRC_H]
```

示例帧：

```
01 03 F0 [240 bytes of 00] 8C DB
```

对应 IMU 数据范围：

```
IMU float[60] ~ IMU float[119]
```

Frame 2：读取 IMU float[120] ~ float[159] 和时间戳

请求帧：

```
[SlaveAddr] [0x03] [0x10 0xF0] [0x00 0x54] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 10 F0 00 54 40 C6
```

含义：

```
起始寄存器：0x10F0  
寄存器数量：0x0054 = 84  
数据长度：84 × 2 = 168 byte = 0xA8  
对应数据：40 个 float32 + 1 个 ROS 时间戳
```

回复帧：

```
[SlaveAddr] [0x03] [0xA8]  
[Reg10F0_H Reg10F0_L]  
[Reg10F1_H Reg10F1_L]  
...  
[Reg1143_H Reg1143_L]  
[CRC_L] [CRC_H]
```

示例帧：

```
01 03 A8 [168 bytes of 00] [CRC_L CRC_H]
```

对应 IMU 数据范围：

```
0x10F0 ~ 0x113F: IMU float[120] ~ IMU float[159], float32, 共 80 regs
0x1140: imu_timestamp.seconds bit[15:0]
0x1141: imu_timestamp.seconds bit[31:16]
0x1142: imu_timestamp.microseconds bit[15:0]
0x1143: imu_timestamp.microseconds bit[31:16]
```

Frame 3: 读取 21 个关节角度和时间戳

请求帧:

```
[SlaveAddr] [0x03] [0x1E 0x00] [0x00 0x2E] [CRC_L] [CRC_H]
```

示例帧:

```
01 03 1E 00 00 2E C3 FE
```

含义:

起始寄存器: 0x1E00
寄存器数量: 0x002E = 46
数据长度: $46 \times 2 = 92$ byte = 0x5C
对应数据: 21 个 joint angle float32 + 1 个 ROS 时间戳

回复帧:

```
[SlaveAddr] [0x03] [0x5C]
[Reg1E00_H Reg1E00_L]
[Reg1E01_H Reg1E01_L]
...
[Reg1E2D_H Reg1E2D_L]
[CRC_L] [CRC_H]
```

示例帧:

```
01 03 5C [92 bytes of 00] [CRC_L CRC_H]
```

对应数据范围:

```

0x1E00 ~ 0x1E29: joint_angle[0] ~ joint_angle[20], float32, 共 42 regs
0x1E2A: joint_timestamp.seconds bit[15:0]
0x1E2B: joint_timestamp.seconds bit[31:16]
0x1E2C: joint_timestamp.microseconds bit[15:0]
0x1E2D: joint_timestamp.microseconds bit[31:16]

```

joint_timestamp 解析为两个独立字段：

```

joint_timestamp.seconds      = ((uint32)Reg1E2B << 16) | Reg1E2A
joint_timestamp.microseconds = ((uint32)Reg1E2D << 16) | Reg1E2C

```

Frame 4：读取电阻点阵 ADC 原始值 R_ADC[0] ~ R_ADC[59]

请求帧：

```
[SlaveAddr] [0x03] [0x20 0x00] [0x00 0x3C] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 20 00 00 3C 4E 1B
```

含义：

```

起始寄存器：0x2000
寄存器数量：0x003C = 60
数据长度：60 × 2 = 120 byte = 0x78
对应数据：60 个 uint16 ADC 原始值

```

回复帧：

```

[SlaveAddr] [0x03] [0x78]
[Reg2000_H Reg2000_L]
[Reg2001_H Reg2001_L]
...
[Reg203B_H Reg203B_L]
[CRC_L] [CRC_H]

```

示例帧：

```
01 03 78 [120 bytes of 00] D1 07
```

对应数据范围：

```
R_ADC[0] ~ R_ADC[59]
```

Frame 5：读取电阻点阵 ADC 原始值 R_ADC[60] ~ R_ADC[119]

请求帧：

```
[SlaveAddr] [0x03] [0x20 0x3C] [0x00 0x3C] [CRC_L] [CRC_H]
```

示例帧：

```
01 03 20 3C 00 3C 8E 17
```

含义：

起始寄存器：0x203C
寄存器数量：0x003C = 60
数据长度：60 × 2 = 120 byte = 0x78
对应数据：60 个 uint16 ADC 原始值

回复帧：

```
[SlaveAddr] [0x03] [0x78]  
[Reg203C_H Reg203C_L]  
[Reg203D_H Reg203D_L]  
...  
[Reg2077_H Reg2077_L]  
[CRC_L] [CRC_H]
```

示例帧：

```
01 03 78 [120 bytes of 00] D1 07
```

对应数据范围：

```
R_ADC[60] ~ R_ADC[119]
```

Frame 6: 读取电阻点阵 ADC 原始值 R_ADC[120] ~ R_ADC[131] 和时间戳

请求帧:

```
[SlaveAddr] [0x03] [0x20 0x78] [0x00 0x10] [CRC_L] [CRC_H]
```

示例帧:

```
01 03 20 78 00 10 CF DF
```

含义:

起始寄存器: 0x2078
寄存器数量: 0x0010 = 16
数据长度: 16 × 2 = 32 byte = 0x20
对应数据: 12 个 uint16 ADC 原始值 + 1 个 ROS 时间戳

回复帧:

```
[SlaveAddr] [0x03] [0x20]  
[Reg2078_H Reg2078_L]  
[Reg2079_H Reg2079_L]  
...  
[Reg2087_H Reg2087_L]  
[CRC_L] [CRC_H]
```

示例帧:

```
01 03 20 [32 bytes of 00] [CRC_L CRC_H]
```

对应数据范围:

0x2078 ~ 0x2083: R_ADC[120] ~ R_ADC[131], uint16, 共 12 regs
0x2084: r_timestamp.seconds bit[15:0]
0x2085: r_timestamp.seconds bit[31:16]

0x2086: r_timestamp.microseconds bit[15:0]
0x2087: r_timestamp.microseconds bit[31:16]

7 帧响应数据长度汇总

帧号	起始寄存器	寄存器数量	ByteCount	数据内容
Frame 0	0x1000	120	0xF0 / 240 byte	IMU float[0] ~ float[59]
Frame 1	0x1078	120	0xF0 / 240 byte	IMU float[60] ~ float[119]
Frame 2	0x10F0	84	0xA8 / 168 byte	IMU float[120] ~ float[159] + ROS timestamp
Frame 3	0x1E00	46	0x5C / 92 byte	joint_angle[0] ~ joint_angle[20] + ROS timestamp
Frame 4	0x2000	60	0x78 / 120 byte	R_ADC[0] ~ R_ADC[59]
Frame 5	0x203C	60	0x78 / 120 byte	R_ADC[60] ~ R_ADC[119]
Frame 6	0x2078	16	0x20 / 32 byte	R_ADC[120] ~ R_ADC[131] + ROS timestamp

7 帧数据区总长度为：

240 + 240 + 168 + 92 + 120 + 120 + 32 = 1012 byte

其中：

- IMU 数据：640 byte
IMU 时间戳：8 byte
关节角度数据：84 byte
关节角度时间戳：8 byte
电阻点阵 ADC 原始值：264 byte
电阻点阵时间戳：8 byte
总计：1012 byte

主机解析说明

主机收到每帧响应后，应按如下顺序处理：

1. 校验 CRC;
2. 检查功能码是否为 0x03;
3. 检查 ByteCount 是否与请求寄存器数量匹配;
4. 提取 Data 区;
5. IMU 和关节角度按 float32 小端寄存器序解析;
6. 电阻点阵按 uint16 ADC 原始值解析, 不转换为 float;
7. 按帧号拼接到对应数据数组。

Frame 0 的数据区长度为 240 byte, 对应 60 个 IMU float:

```
float_count = ByteCount / 4 = 240 / 4 = 60  
imu_data[0] ~ imu_data[59]
```

Frame 1 写入:

```
imu_data[60] ~ imu_data[119]
```

Frame 2 写入:

```
imu_data[120] ~ imu_data[159]  
imu_timestamp.seconds  
imu_timestamp.microseconds
```

Frame 3 写入:

```
joint_angle[0] ~ joint_angle[20]  
joint_timestamp.seconds  
joint_timestamp.microseconds
```

Frame 4 写入:

```
r_adc[0] ~ r_adc[59]
```

Frame 5 写入:

```
r_adc[60] ~ r_adc[119]
```

Frame 6 写入:


```
r_adc[120] ~ r_adc[131]
r_timestamp.seconds
r_timestamp.microseconds
```

9.4 主机解析任务

主机收到 7 帧后：

1. 校验每帧 CRC；
2. 根据帧号拼接 IMU 数据、关节角度和电阻点阵 ADC 原始值；
3. 按 float32 小端寄存器序解析 160 个 IMU float；
4. 按 float32 小端寄存器序解析 21 个关节角度 float；
5. 按 ROS 格式解析 IMU、关节角度和电阻点阵时间戳：seconds uint32 + microseconds uint32；
6. 按 uint16 解析 132 个电阻点阵 ADC 原始值；
7. 更新上位机显示和算法输入。

10. 低频状态监控任务

在高频数据交互过程中，主机还应低频读取关键状态。建议频率如下：

- 1 Hz：
- 读取 SD 卡状态；
 - 读取错误诊断；
 - 读取电池电压、电流；
 - 读取系统状态。
- 10 Hz：
- 读取 IMU 状态位；
 - 读取电阻点阵状态位；
 - 读取当前 UTC 时间戳。

低频读取内容：

SD 状态：
0x0081 ~ 0x00BF

系统状态：
0x0040 ~ 0x0049

电源状态：
0x0060 ~ 0x0063

IMU 状态：
0x1144

电阻点阵状态：
0x2088 ~ 0x2090

若出现以下情况，主机应停止采集或报警：

SD 写入异常；
SD 空间不足；
电池电压过低；
电流异常；
IMU 状态位异常；
电阻点阵状态位异常；
RS485 CRC 或超时错误过多；
时间同步失败。

11. 主机完整任务流程总结

1. 地址发现

- 主机使用 0x00 发送地址发现帧；
- 读取 0x0000，即 REG_SLAVE_ADDR；
- 从机回复帧地址保持 0x00，REG_SLAVE_ADDR 数据区返回真实从机地址；
- 主机从寄存器值记录 slave_addr。

2. 设备状态查询

- 读取基础通信与时间寄存器；
- 读取系统状态；
- 读取电源状态；
- 读取 SD 卡与日志状态；
- 读取 IMU 状态；
- 读取电阻点阵状态。

3. 时间同步

- 主机获取 UTC；
- 主机触发 GPIO 同步脉冲；
- 主机写入 UTC 时间戳；
- 从机建立同步基准；
- 主机读取 UTC 或状态确认同步成功。

4. IMU 与关节角度校准

- 主机语音提示用户执行动作；
- 主机持续读取 IMU 高频数据；
- 主机读取或写入 21 个关节角度 offset；
- 每个动作完成后在本地记录校准数据；
- 计算 IMU Offset 并写入 IMU Offset 寄存器区。

5. 电阻点阵校准

- 用户保持无压力状态；
- 主机读取电阻点阵 ADC 原始值；

- 主机在本地计算并保存 132 个电阻点的零点基准。
6. 开启 SD 记录
 - 主机发送创建日志文件命令；
 - 主机查询 SD 状态；
 - 主机发送开始日志记录命令；
 - 主机确认日志状态为正在记录。
 7. 采集控制任务
 - 主机写入 `REG\_CMD = CMD\_ACQ\_START 0x0501`；
 - 主机读取 `REG\_WORK\_STATE 0x0500`；
 - 工作状态为正在采集后进入高频采集通信；
 - 采集结束时主机写入 `REG\_CMD = CMD\_ACQ\_STOP 0x0502`；
 - 工作状态为空闲后退出高频采集通信。
 8. 高频数据交互
 - 主机以 100 Hz 周期读取 7 帧数据；
 - 解析 IMU float 数据；
 - 解析 21 个关节角度 float 数据；
 - 解析 132 个电阻点阵 uint16 ADC 原始值；
 - 低频查询 SD、电源、错误和时间同步状态。

12. 从机任务处理原则

1. 地址发现请求：
 - 若收到地址 0x00，功能码 0x03，读取 0x0000，数量 1；
 - 从机回复帧地址保持 0x00，并在 REG_SLAVE_ADDR 数据区返回真实地址。
2. 读寄存器请求：
 - 根据起始地址判断属于哪个区域；
 - uint16 直接返回；
 - float32 按小端寄存器序打包；
 - uint64 按小端寄存器序打包。
3. 写 UTC 请求：
 - 主机写入 REG_TIME_SYNC_UTC；
 - 从机结合 GPIO 同步中断记录的本地计时零点建立 UTC 基准。
4. 写控制寄存器请求：
 - 主机写 REG_CMD、REG_CMD_PARAM、REG_CMD_SEQ 时，从机根据采集控制、日志控制、SD 文件列表、文件下载等命令执行对应任务；
 - 收到 `CMD\_ACQ\_START 0x0501` 时，从机启动采集任务并更新 `REG\_WORK\_STATE 0x0500`；
 - 收到 `CMD\_ACQ\_STOP 0x0502` 时，从机结束采集任务并更新 `REG\_WORK\_STATE 0x0500`；
 - 收到 `CMD\_LOG\_START 0x0094` / `CMD\_LOG\_STOP 0x0096` 时，从机开始或停止日志记录，并更新 `REG\_SD\_LOG\_STATUS 0x0082`；
 - 命令寄存器任务执行完成后更新 REG_CMD_ACK。
5. 高频数据请求：
 - IMU 数据直接从 IMU float 数组指定位置打包；
 - 关节角度数据直接从 joint angle float 数组指定位置打包；

- 电阻点阵数据直接从 `uint16` ADC 原始值数组指定位置打包；
- 使用双缓冲或临界区保证数据一致性。

6. SD 日志任务：
- 采集开始后，如果日志状态为正在记录，则写入 SD；
 - 实时更新文件大小、写入帧数、日志状态和错误码。

附录 A. Holding Register 对照表

本附录用于和正文任务流程中的寄存器地址进行对照。每个 Holding Register 为 16 bit；`float32` 占 2 个寄存器，`uint64` 占 4 个寄存器，均采用小端寄存器序；`ros_timestamp` 占 4 个寄存器，其中 seconds `uint32` 占 2 个寄存器、microseconds `uint32` 占 2 个寄存器；单个寄存器内部仍采用标准 Modbus 大端字节序。

A.1 基础通信与时间寄存器区

地址	名称	类型	说明
<code>0x0000</code>	<code>REG_SLAVE_ADDR</code>	<code>uint16</code>	从站地址
<code>0x0001</code>	<code>REG_BAUDRATE_CODE</code>	<code>uint16</code>	波特率编码
<code>0x0002 ~ 0x0005</code>	<code>REG_UTC_TIMESTAMP</code>	<code>ros_timestamp</code>	当前系统 UTC 时间戳：seconds <code>uint32</code> + microseconds <code>uint32</code>
<code>0x0006 ~ 0x0009</code>	<code>REG_LOCAL_UPTIME_MS</code>	<code>uint64</code>	本地运行时间，单位 us
<code>0x000A ~ 0x000D</code>	<code>REG_TIME_SYNC_UTC</code>	<code>ros_timestamp</code>	主机写入的同步 UTC：seconds <code>uint32</code> + microseconds <code>uint32</code>

A.2 命令寄存器区

命令寄存器区统一放在 `0x0020 ~ 0x003E`，用于主机向从机下发采集控制、日志控制、SD 文件列表、文件下载等扩展控制命令。IMU 校准和电阻点阵校准不通过命令寄存器触发。

地址	名称	类型	说明
<code>0x0020</code>	<code>REG_CMD</code>	<code>uint16</code>	命令字
<code>0x0021</code>	<code>REG_CMD_PARAM</code>	<code>uint16</code>	命令参数
<code>0x0022</code>	<code>REG_CMD_SEQ</code>	<code>uint16</code>	命令序号
<code>0x0023</code>	<code>REG_CMD_ACK</code>	<code>uint16</code>	命令执行应答
<code>0x0024</code>	<code>REG_CMD_ACK_SEQ</code>	<code>uint16</code>	从机已处理命令序号，回显最近一次处理的 <code>REG_CMD_SEQ</code>
<code>0x0025</code>	<code>REG_CMD_ERROR</code>	<code>uint16</code>	命令执行错误码或补充错误原因
<code>0x0026 ~ 0x003E</code>	<code>REG_CMD_RESERVED</code>	<code>uint16[]</code>	命令区保留，后续可扩展命令状态、错误码或多参数

命令字编码统一写入 `REG_CMD`，即 `0x0020`。正文中需要通过命令寄存器下发的命令字统一如下：

命令字	数值	REG_CMD_PARAM	说明
CMD_NONE	0x0000	0	无命令
CMD_LOG_START	0x0094	0	开始日志记录
CMD_LOG_STOP	0x0096	0	停止日志记录
CMD_ACQ_START	0x0501	0	开始采集任务
CMD_ACQ_STOP	0x0502	0	结束采集任务
CMD_SD_LIST_FILES	待重新分配	page_id	读取 SD 文件列表，page_id 从 0 开始
CMD_SD_DOWNLOAD_FILE	待重新分配	file_id	下载指定日志文件

`REG_CMD_ACK` 用于返回命令执行状态：

ACK 值	说明
0x0000	空闲或尚未执行
0x0001	命令执行成功
0x0002	命令执行中
0x8001	未知命令
0x8002	参数错误
0x8003	当前状态不允许执行
0x8004	命令执行失败

`REG_CMD_ACK_SEQ` 与 `REG_CMD_ERROR` 约定：

REG_CMD_ACK_SEQ：
从机处理完某条命令后，写入该命令对应的 `REG_CMD_SEQ`。
主机读取 `ACK` 时必须检查 `ACK_SEQ` 是否等于本次命令 `seq`。

REG_CMD_ERROR：
当 `REG_CMD_ACK` 为错误状态时，返回更具体的错误原因。
当 `REG_CMD_ACK` = `0x0001` 时，`REG_CMD_ERROR` = `0x0000`。

命令错误码建议：

ERROR 值	说明
0x0000	无错误
0x0001	命令参数无效
0x0002	命令序号重复或过期

ERROR 值	说明
0x0003	当前工作状态不允许执行
0x0004	资源未就绪，例如 SD 未挂载
0x0005	任务启动失败
0x0006	任务停止失败
0x0007	内部超时

常用帧格式：

```
写命令：
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [0x06]
[CMD_H CMD_L]
[PARAM_H PARAM_L]
[SEQ_H SEQ_L]
[CRC_L] [CRC_H]

写命令回复：
[SlaveAddr] [0x10] [0x00 0x20] [0x00 0x03] [CRC_L] [CRC_H]

读 ACK：
[SlaveAddr] [0x03] [0x00 0x23] [0x00 0x03] [CRC_L] [CRC_H]
```

示例帧：

```
写命令回复： 01 10 00 20 00 03 81 C2
读 ACK： 01 03 00 23 00 03 F4 01
```

ACK 回复数据区：

```
[REG_CMD_ACK_H REG_CMD_ACK_L]
[REG_CMD_ACK_SEQ_H REG_CMD_ACK_SEQ_L]
[REG_CMD_ERROR_H REG_CMD_ERROR_L]
```

A.3 工作状态区

地址	名称	类型	说明
0x0500	REG_WORK_STATE	uint16	工作状态：空闲、正在采集、正在停止或异常

工作状态值：

状态值	名称	说明
0x0000	WORK_STATE_IDLE	空闲
0x0001	WORK_STATE_ACQUIRING	正在采集
0x0002	WORK_STATE_STOPPING	正在停止采集
0x8000	WORK_STATE_ERROR	采集异常

A.4 系统状态区

地址	名称	类型	说明
0x0040	REG_SYSTEM_STATE	uint16	系统状态
0x0041	REG_WORK_MODE	uint16	工作模式
0x0042	REG_LOG_STATE	uint16	日志状态
0x0043	REG_SD_STATE	uint16	SD 卡总状态
0x0044	REG_SENSOR_STATE	uint16	传感器总状态位
0x0045	REG_COMM_STATE	uint16	RS485 通信状态
0x0046 ~ 0x0047	REG_SYSTEM_RESERVED	uint16[2]	保留
0x0048 ~ 0x0049	REG_TEMPERATURE_BOARD	float32	板载温度

A.5 电源状态区

地址	名称	类型	说明
0x0060 ~ 0x0061	REG_BAT_VOLTAGE	float32	电池电压，单位 V
0x0062 ~ 0x0063	REG_BAT_CURRENT	float32	电池电流，单位 A

A.6 SD 卡与日志文件状态区

地址	名称	类型	说明
0x0081	REG_SD_FS_STATUS	uint16	文件系统状态
0x0082	REG_SD_LOG_STATUS	uint16	日志记录状态
0x0083	REG_SD_ERROR_CODE	uint16	SD/FatFs 最近一次错误码
0x0084 ~ 0x0085	REG_SD_TOTAL_SIZE_MB	uint32	SD 卡总容量，单位 MB
0x0086 ~ 0x0087	REG_SD_FREE_SIZE_MB	uint32	SD 卡剩余容量，单位 MB
0x0088 ~ 0x0089	REG_SD_USED_SIZE_MB	uint32	SD 卡已用容量，单位 MB

地址	名称	类型	说明
0x008A	REG_SD_CURRENT_FILE_ID	uint16	当前日志文件编号
0x008C ~ 0x008F	REG_SD_CURRENT_FILE_SIZE	uint64	当前日志文件大小，单位 byte
0x0090 ~ 0x0091	REG_SD_CURRENT_WRITE_CNT	uint32	当前文件已写入帧数
0x0092	CMD_LOG_CREATE_FILE	uint16	写入触发从机创建新的日志文件，读取返回创建成功状态
0x0093	REG_SD_RESERVED	uint16	保留
0x0094	REG_SD_DISK_LAST_RESULT	uint16	最近一次 FatFs diskio DRESULT
0x0095	REG_SD_DISK_HAL_STATUS	uint16	最近一次 HAL_SD 操作返回状态
0x0096 ~ 0x0097	REG_SD_DISK_HAL_ERROR	uint32	最近一次 HAL_SD 错误码
0x0098 ~ 0x0099	REG_SD_RESERVED	uint16[2]	保留
0x009A ~ 0x009D	CMD_LOG_LENGTH	uint64	查询指定文件长度，读取返回文件长度
0x009E ~ 0x009F	REG_SD_RESERVED	uint16[2]	保留
0x00A0 ~ 0x00AF	REG_SD_CURRENT_FILENAME	uint16[16]	当前日志文件名
0x00B0 ~ 0x00BF	REG_SD_LAST_FILENAME	uint16[16]	上一个日志文件名

说明：0x0094 和 0x0096 在这里是 SD 状态区的寄存器地址；CMD_LOG_START 0x0094 / CMD_LOG_STOP 0x0096 是写入命令寄存器 REG_CMD 0x0020 的命令值，二者含义不同。

A.7 IMU 数据、时间戳、状态与 Offset 区

地址	名称	类型	说明
0x1000 ~ 0x113F	REG_IMU_DATA_START ~ REG_IMU_DATA_END	float32[160]	16 个 IMU，每个 IMU 10 个 float32
0x1140 ~ 0x1143	REG_IMU_TIMESTAMP	ros_timestamp	当前 IMU 数据帧时间戳： seconds uint32 + microseconds uint32
0x1144	REG_IMU_STATUS_BITS	uint16	IMU 状态位，bit0~bit15 对应 IMU0~IMU15

地址	名称	类型	说明
0x1154 ~ 0x1293	REG_IMU_OFFSET_START ~ REG_IMU_OFFSET_END	float32[160]	16 个 IMU 的初始 Offset，每个 IMU 10 个 float32

IMU 状态位含义：

bit = 1: 对应 IMU 正常
bit = 0: 对应 IMU 异常或离线

IMU 数据区按每个 IMU 20 个寄存器排列：

REG_IMU0_DATA_START	0x1000	REG_IMU0_DATA_END	0x1013
REG_IMU1_DATA_START	0x1014	REG_IMU1_DATA_END	0x1027
REG_IMU2_DATA_START	0x1028	REG_IMU2_DATA_END	0x103B
REG_IMU3_DATA_START	0x103C	REG_IMU3_DATA_END	0x104F
REG_IMU4_DATA_START	0x1050	REG_IMU4_DATA_END	0x1063
REG_IMU5_DATA_START	0x1064	REG_IMU5_DATA_END	0x1077
REG_IMU6_DATA_START	0x1078	REG_IMU6_DATA_END	0x108B
REG_IMU7_DATA_START	0x108C	REG_IMU7_DATA_END	0x109F
REG_IMU8_DATA_START	0x10A0	REG_IMU8_DATA_END	0x10B3
REG_IMU9_DATA_START	0x10B4	REG_IMU9_DATA_END	0x10C7
REG_IMU10_DATA_START	0x10C8	REG_IMU10_DATA_END	0x10DB
REG_IMU11_DATA_START	0x10DC	REG_IMU11_DATA_END	0x10EF
REG_IMU12_DATA_START	0x10F0	REG_IMU12_DATA_END	0x1103
REG_IMU13_DATA_START	0x1104	REG_IMU13_DATA_END	0x1117
REG_IMU14_DATA_START	0x1118	REG_IMU14_DATA_END	0x112B
REG_IMU15_DATA_START	0x112C	REG_IMU15_DATA_END	0x113F

单个 IMU Offset 起始地址计算公式：

IMU_i_OFFSET_BASE = 0x1154 + i × 20
i = 0 ~ 15

REG_IMU_OFFSET_ADDR(i, field_offset) = IMU_i_OFFSET_BASE + field_offset

单个 IMU Offset 字段地址偏移：

字段	地址偏移	寄存器数量	数据类型	说明
qw_offset	+0	2	float32	四元数 w 初始值
qx_offset	+2	2	float32	四元数 x 初始值
qy_offset	+4	2	float32	四元数 y 初始值
qz_offset	+6	2	float32	四元数 z 初始值

字段	地址偏移	寄存器数量	数据类型	说明
acc_x_offset	+8	2	float32	X 轴加速度初始值
acc_y_offset	+10	2	float32	Y 轴加速度初始值
acc_z_offset	+12	2	float32	Z 轴加速度初始值
gyro_x_offset	+14	2	float32	X 轴角速度初始值
gyro_y_offset	+16	2	float32	Y 轴角速度初始值
gyro_z_offset	+18	2	float32	Z 轴角速度初始值

IMU Offset 区按每个 IMU 20 个寄存器排列：

REG_IMU0_OFFSET_START	0x1154	REG_IMU0_OFFSET_END	0x1167
REG_IMU1_OFFSET_START	0x1168	REG_IMU1_OFFSET_END	0x117B
REG_IMU2_OFFSET_START	0x117C	REG_IMU2_OFFSET_END	0x118F
REG_IMU3_OFFSET_START	0x1190	REG_IMU3_OFFSET_END	0x11A3
REG_IMU4_OFFSET_START	0x11A4	REG_IMU4_OFFSET_END	0x11B7
REG_IMU5_OFFSET_START	0x11B8	REG_IMU5_OFFSET_END	0x11CB
REG_IMU6_OFFSET_START	0x11CC	REG_IMU6_OFFSET_END	0x11DF
REG_IMU7_OFFSET_START	0x11E0	REG_IMU7_OFFSET_END	0x11F3
REG_IMU8_OFFSET_START	0x11F4	REG_IMU8_OFFSET_END	0x1207
REG_IMU9_OFFSET_START	0x1208	REG_IMU9_OFFSET_END	0x121B
REG_IMU10_OFFSET_START	0x121C	REG_IMU10_OFFSET_END	0x122F
REG_IMU11_OFFSET_START	0x1230	REG_IMU11_OFFSET_END	0x1243
REG_IMU12_OFFSET_START	0x1244	REG_IMU12_OFFSET_END	0x1257
REG_IMU13_OFFSET_START	0x1258	REG_IMU13_OFFSET_END	0x126B
REG_IMU14_OFFSET_START	0x126C	REG_IMU14_OFFSET_END	0x127F
REG_IMU15_OFFSET_START	0x1280	REG_IMU15_OFFSET_END	0x1293

A.8 关节角度、时间戳与状态区

地址	名称	类型	说明
0x1E00 ~ 0x1E29	REG_JOINT_ANGLE_START ~ REG_JOINT_ANGLE_END	float32[21]	21 个关节角度
0x1E2A ~ 0x1E2D	REG_JOINT_TIMESTAMP	ros_timestamp	当前关节角度数据帧时间戳：seconds uint32 + microseconds uint32
0x1E2E ~ 0x1E2F	REG_JOINT_ANGLE_STATUS_START ~ REG_JOINT_ANGLE_STATUS_END	uint16[2]	当前先用 2 个寄存器表示 21 个关节角度状态
0x1E30 ~ 0x1E59	REG_JOINT_ANGLE_CALIB_START ~ REG_JOINT_ANGLE_CALIB_END	float32[21]	21 个校准后的关节角度校准值
0x1E5A ~ 0x1FFF	REG_JOINT_RESERVED	uint16[]	关节角度区扩展保留

关节角度 ROS 时间戳寄存器排列：

```
REG_JOINT_TIMESTAMP_SEC_L      0x1E2A
REG_JOINT_TIMESTAMP_SEC_H      0x1E2B
REG_JOINT_TIMESTAMP_USEC_L     0x1E2C
REG_JOINT_TIMESTAMP_USEC_H     0x1E2D

joint_timestamp.seconds        = ((uint32)REG_JOINT_TIMESTAMP_SEC_H  << 16) |
REG_JOINT_TIMESTAMP_SEC_L
joint_timestamp.microseconds    = ((uint32)REG_JOINT_TIMESTAMP_USEC_H << 16) |
REG_JOINT_TIMESTAMP_USEC_L
```

ROS 时间戳是 `seconds` 与 `microseconds` 两个独立 `uint32` 字段，不按单个 `uint64` 打包或比较。

关节角度状态当前使用 2 个寄存器按 bit 表示：

```
0x1E2E: joint_angle[0] ~ joint_angle[15] 状态位
0x1E2F: joint_angle[16] ~ joint_angle[20] 状态位, bit5 ~ bit15 保留为 0
bit = 1: 对应关节角度有效
bit = 0: 对应关节角度无效、异常或未更新
```

关节角度校准值区按每个校准值 2 个寄存器排列：

```
REG_JOINT_ANGLE_CALIB0_START  0x1E30    REG_JOINT_ANGLE_CALIB0_END  0x1E31
REG_JOINT_ANGLE_CALIB1_START  0x1E32    REG_JOINT_ANGLE_CALIB1_END  0x1E33
...
REG_JOINT_ANGLE_CALIB20_START 0x1E58    REG_JOINT_ANGLE_CALIB20_END 0x1E59
```

A.9 电阻点阵 ADC、时间戳与状态区

地址	名称	类型	说明
0x2000 ~ 0x2083	REG_R_ADC_START ~ REG_R_ADC_END	uint16[132]	132 个电阻点 ADC 原始值
0x2084 ~ 0x2087	REG_R_TIMESTAMP	ros_timestamp	当前电阻点阵数据帧时间戳：seconds uint32 + microseconds uint32
0x2088 ~ 0x2090	REG_R_STATUS_START ~ REG_R_STATUS_END	uint16[9]	132 个电阻点状态位
0x2091 ~ 0x20C3	REG_R_STATUS_RESERVED_START ~ REG_R_STATUS_RESERVED_END	uint16[]	电阻点阵状态扩展保留区

电阻点阵 ADC 原始值区按每个电阻点 1 个寄存器排列：

```
REG_R_ADC0      0x2000
REG_R_ADC1      0x2001
...
REG_R_ADC131    0x2083
```

电阻点阵状态位含义：

```
bit = 1: 对应电阻点正常
bit = 0: 对应电阻点异常、离线或数据无效

0x2088: R0    ~ R15    状态位
0x2089: R16   ~ R31    状态位
0x208A: R32   ~ R47    状态位
0x208B: R48   ~ R63    状态位
0x208C: R64   ~ R79    状态位
0x208D: R80   ~ R95    状态位
0x208E: R96   ~ R111   状态位
0x208F: R112  ~ R127   状态位
0x2090: R128  ~ R131   状态位, bit4 ~ bit15 保留为 0
```